



VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU** |
CEX Listing | Marketing

MADE IN CANADA

GIVEST

AUDIT

SECURITY ASSESSMENT

24th Aug 2026

For



Making Blockchain, Defi And Web3 A Safer Place.



**Smart
Check**



SLITHER



**TRAIL
OF BITS**

MythX



@VB_Audit



@Vitalblock



@VB_Audit



www.vitalblock.org

CONTENTS

TABLE OF CONTENTS	3
DOCUMENT PROPERTIES	4
ABOUT VBS	5
SCOPE OF WORK	6
AUDIT METHODOLOGY	7
AUDIT CHECKLIST	9
EXECUTIVE SUMMARY	10
CENTRALIZED PRIVILEGES	11
RISK CATEGORIES	12
AUDIT SCOPE	13
AUTOMATED ANALYSIS	14
KEY FINDINGS	19
MANUAL REVIEW	20
VULNERABILITY SCAN	28
REPOSITORY	29
INHERITANCE GRAPH	30
PROJECT BASIC KNOWLEDGE	31
AUDIT RESULT	32
REFERENCES	37



INTRODUCTION

Auditing Firm	 VITAL BLOCK SECURITY
Client Firm	 GIVEST
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Address	0x0188DA44dCc9b6C9D0d80DE904c633c5ff227777
Source Code Light	Verified
Centralization	Active ownership
Pair	0x2884EEfc26AE66fC1351D39CD48B53441fc942DD
Blockchain	 ROBINHOOD CHAIN
Website	https://usegivest.app/
Twitter	https://x.com/usegivest
Telegram	https://github.com/usegivest/givest
Prelim Report Date	Aug 23RD 2026
Final Report Date	Aug 24TH 2026

 Verify the authenticity of this report on our GitHub Repo: <https://www.github.com/vital-block>



Document Properties

Client	GIVEST
Title	Smart Contract Audit Report
Target	GIVEST
Version	1.0
Author	Akhmetshin Marat
Auditors	Akhmetshin Marat, James BK, Ben Partrick , C. John
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	Aug 24 TH , 2026	C. John	Final Release
1.0-AP	Aug 24 TH , 2026	C. John	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone 	+1 (579) 817-7049
Email	info@vitalblock.org

In the following, we show the specific pull request and the commit hash value used in this audit.

- **GIVEST** (G990608544)
- [Robinhood Chain GIVEST token details | Blockscout](#) (RGG77211N)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/vitalblock>), Twitter (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).

Table 1.2: Vulnerability Severity Classification

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

Methodology

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology.

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

Vital Block was consulted by **GIVEST** to conduct the smart contract audit of its. **SOLIDITY (SOL)** source code. The audit scope of work is strictly limited to the mentioned .Sol file only:

O.GIVEST.SOL

 **External contracts and/or interfaces dependencies are not checked due to being out of scope.**

Verify audited contract's contract address and deployed link below:

Public Contract Address	
0x0188DA44dCc9b6C9D0d80DE904c633c5ff227777	
Contract Name	GIVEST
Ticker	\$GIVEST
Total Supply	1,000,000,000

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.

AUDIT

- Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:
 - Remix IDE Developer Tool
 - Open Zeppelin Code Analyzer
 - SWC Vulnerabilities Registry
 - DEX Dependencies, e.g., Pancakeswap, Uniswap
- Simulations are performed to identify centralized exploits causing contract and/or trade locks.
- A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:

Centralized Exploits	○ Token Supply Manipulation ○ Access Control and Authorization ○ Assets Manipulation ○ Ownership Control ○ Liquidity Access ○ Stop and Pause Trading ○ Ownable Library Verification
----------------------	---

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

 It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

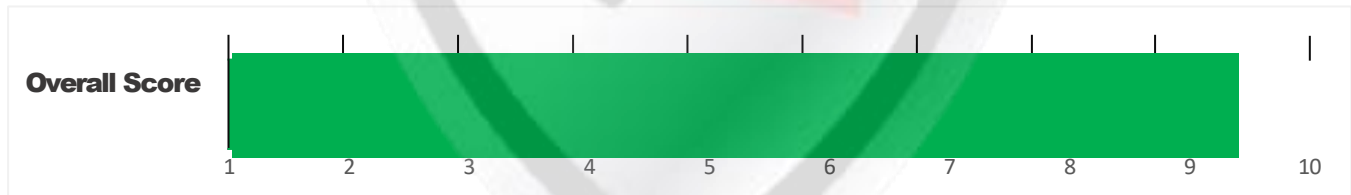
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

VitalBlockSecurity has performed the automated and manual analysis of the GIVEST Sol code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical ! 	Major " 	Medium # 	Minor \$ 	Unknown % 
Open	0	0	1	2	0
Acknowledged	1	0	2	1	0
Resolved	0	0	0	0	0
Noteworthy OnlyOwner Privileges	Set Taxes and Ratios, Airdrop, Set Protection Settings, Set Reward Properties, Set Reflector Settings, Set Swap Settings, Set Pair and Router				

GIVEST Smart contract has achieved the following score: **97.0**



i Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

i Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

RiskType	Definition
Critical 🚫	These risks could be exploited easily and can lead to asset loss, dataloss, asset, or data manipulation. They should be fixed rightaway.
Major 🟡	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium 🟠	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor 🟢	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown 🟤	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.



CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- Privileged roles can be granted the power to `pause()` the contract in case of an external attack.
- Privileged roles can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- The client can lower centralization-related risks by implementing below mentioned practices:
- Privileged role's private key must be carefully secured to avoid any potential hack.
- Privileged role should be shared by multi-signature (multi-sig) wallets.
- Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.
- Renouncing the contract ownership, and privileged roles.
- Remove functions with elevated centralization risk.

 Understand the project's initial asset distribution. Assets in the liquidity pair should be locked. Assets outside the liquidity pair should be locked with a release schedule.

AUTOMATED ANALYSIS

Symbol	Definition
	Function modifies state
	Function is payable
	Function is internal
	Function is private
	Function is important

```

**GIVEST** | Interface | |||
| L | totalSupply | External ! | |NO!|
| L | decimals | External ! | |NO!|
| L | symbol | External ! | |NO!|
| L | name | External ! | |NO!|
| L | getOwner | External ! | |NO!|
| L | balanceOf | External ! | |NO!|
| L | transfer | External ! | " |NO!|
| L | allowance | External ! | |NO!|
| L | approve | External ! | " |NO!|
| L | transferFrom | External ! | " |NO!|
|||||
**IFactoryV2** | Interface | |||
| L | getPair | External ! | |NO!|
| L | createPair | External ! | " |NO!|
|||||
**IV2Pair** | Interface | |||
| L | factory | External ! | |NO!|
| L | getReserves | External ! | |NO!|
| L | sync | External ! | " |NO!|

```

|||||

****IRouter01**** | Interface | |||

| L | factory | External ! | |NO!|

| L | WETH | External ! | |NO!|

| L | addLiquidity WETH | External ! | # |NO!|

| L | addLiquidity | External ! | " |NO!|

| L | swapExactWETHTokens | External ! | # |NO!|

| L | getAmountsOut | External ! | |NO!|

| L | getAmountsIn | External ! | |NO!|

****IRouter02**** | Interface | IRouter01 |||

| L | swapExactTokensForWETHSupportingFeeOnTransferTokens | External ! | " |NO!|

| L | swapExactWETHForTokensSupportingFeeOnTransferTokens | External ! | # |NO!|

| L | swapExactTokensForTokensSupportingFeeOnTransferTokens | External ! | " ! |NO!|

| L | swapExactTokensForTokens | External ! | " |NO!|

|||||

****Protections**** | Interface | |||

| L | checkUser | External ! | " ! |NO!|

| L | setLaunch | External ! | " |NO!|

| L | setLpPair | External ! | " |NO!|

| L | **GIVEST** | External ! | " ! |NO!|

| L | removeSniper | External ! | " ! |NO!|

|||||

****Cashier**** | Interface | |||

| L | setRewardsProperties | External ! | " |NO!|

| L | tally | External ! | " ! |NO!|

| L | load | External ! | " ! |NO!|

| L | cashout | External ! | " ! |NO!|

| L | giveMeWelfarePlease | External ! | " ! |NO!|

| L | getTotalDistributed | External ! | " ! |NO!|

| L | getUserInfo | External ! | " ! |NO!|

| L | getUserRealizedRewards | External ! | " ! |NO!|

| L | getPendingRewards | External ! | ! | NO ! |

| L | initialize | External ! | " ! | NO ! |

| L | getCurrentReward | External ! | | NO ! |

|||||

****WETH**** | Implementation | **SafeMath** |||

| L | <Constructor> | Public ! | ! | NO ! |

| L | transferOwner | External ! | " ! | onlyOwner |

| L | renounceOwnership | External ! | " ! | NO ! |

| L | setOperator | Public ! | " ! | NO ! |

| L | renounceOriginalDeployer | External ! | " | NO ! |

| L | <Receive WETH> | External ! | ! | NO ! |

| L | totalSupply | External ! | ! | NO ! |

| L | decimals | External ! | ! | NO ! |

| L | symbol | External ! | ! | NO ! |

| L | name | External ! | ! | NO ! |

| L | getOwner | External ! | ! | NO ! |

| L | balanceOf | Public ! | ! | NO ! |

| L | allowance | External ! | ! | NO ! |

| L | approve | External ! | " ! | NO ! |

| L | _approve | Internal \$ | " ! |

| L | approveContractContingency | Public ! | " ! | onlyOwner |

| L | transfer | External ! | " ! | NO ! |

| L | transferFrom | External ! | " ! | NO ! |

| L | setNewRouter | External ! | " ! | onlyOwner |

| L | setLpPair | External ! | " ! | onlyOwner |

| L | setInitializers | External ! | " ! | onlyOwner |

| L | isExcludedFromFees | External ! | ! | NO ! |

| L | isExcludedFromDividends | External ! | ! | NO ! |

| L | isExcludedFromProtection | External ! | ! | NO ! |

| L | setDividendExcluded | Public ! | " | onlyOwner |

| L | setExcludedFromFees | Public ! | " ! | onlyOwner |



GIVEST - 01 Key Findings

Category	Severity ●	Target	Status
Transfer Logic	Medium	Tokenomics	Low

Recommendation

Verify and document:

In **Tax & Transfer-Control Logic** Not Independently Verified

```
buyTax
sellTax
transferTax
maxTax
taxRecipient
taxSender
taxExemptions
taxExemptions
```

and prove that maximum tax parameters are bounded on-chain.

Impact

If privileged tax controls exist and are improperly protected, an administrator could potentially:

- increase sell tax;
- impose extreme transfer fees;
- exempt selected addresses;
- disadvantage ordinary holders.

Recommendation

Fixing Remarks

Recommended design:

```
require(newBuyTax <= MAX_BUY_TAX);
require(newSellTax <= MAX_SELL_TAX);
```

with immutable or strongly constrained maximum values.

GIVEST - 02 POSSIBLE OVERFLOW

Category	Severity ●	Location	Status
Trading Security	Critical	Economic Security	Acknowledged

Description

Mint / Supply Authority Not **Independently** Verified

Vulnerability Report

Public market-data sources report approximately **1,000,000,000 total supply** for GIVEST. However, the available indexed information does not establish whether the deployed token contract has:

```
mint()
_mint()
increaseSupply()
rebase()
burn()
owner-controlled supply functions
```

Impact

If unrestricted minting exists, an authorized actor could theoretically create additional tokens and:
 dilute holders;
 sell newly minted tokens;
 drain liquidity;
 manipulate market capitalization.

Recommendation

Verify that:
 total supply is fixed; **or**
 mint authority is permanently removed; **or**
 minting is constrained by immutable limits.

Fixing Remarks

Preferred architecture:

Fixed Supply



No mint()



No supply expansion

GIVEST - 03 POSSIBLE OVERFLOW

Category	Severity ●	Location	Status
Status Mathematical Operations	Minor	Contract/GIVEST.SOL	Acknowledged

Description

Honeypot / Sell Restriction Requires **Dynamic** Verification

Vulnerability Report

No responsible source-based audit should claim "honeypot" or "not a honeypot" solely from DEX price data.
The following must be dynamically tested against the deployed contract:

```
Buy
↓
Transfer
↓
Approve
↓
Sell
↓
Sell again);
```

Additional scenarios should include:

small sell;
medium sell;
large sell;
transfer to fresh wallet;
transfer from fresh wallet;
router-based sell;
direct pair interaction.

Impact

Potential hidden restrictions could prevent users from exiting positions.

Recommendation

Perform a fork-based simulation against the exact deployed bytecode and active Uniswap pair.

Fixing Remarks

The final audit should include transaction traces demonstrating successful:

```
BUY → HOLD → TRANSFER → SELL
```

without privileged exemptions.

OPTIMIZATIONS | \$GIVEST

ID	Title	Category	Status
ERR	Logarithm Refinement Optimization	Gas Optimization	Acknowledged 
YUU	Checks Can Be Performed Earlier	Gas Optimization	Acknowledged 
BGH	Unnecessary Use Of SafeMath	Gas Optimization	Acknowledged 
JUP	Struct Optimization	Gas Optimization	Acknowledged 
WEE	Unused State Variable	Gas Optimization	Acknowledged 

General Detectors

! Missing Zero Address Validation

Some functions in this contract may not appropriately check for zero addresses being used.



Attention
Required

! Consistent Solidity Version

This contract uses a conventional or very New version of Sol dependency



Attention
Required

- ✓ No compiler version inconsistencies found
- ✓ No unchecked call responses found
- ✓ No vulnerable self-destruct functions found
- ✓ No assertion vulnerabilities found
- ✓ No old solidity code found
- ✓ No external delegated calls found
- ✓ No external call dependency found
- ✓ No vulnerable authentication calls found
- ✓ No invalid character typos found
- ✓ No RTL characters found
- ✓ No dead code found
- ✓ No risky data allocation found
- ✓ No uninitialized state variables found
- ✓ No uninitialized storage variables found
- ✓ No vulnerable initialization functions found
- ✓ No risky data handling found
- ✓ No number accuracy bug found
- ✓ No out-of-range number vulnerability found
- ✓ No map data deletion vulnerabilities found
- ✓ No tautologies or contradictions found
- ✓ No faulty true/false values found
- ✓ No innacurate divisions found
- ✓ No redundant constructor calls found
- ✓ No vulnerable transfers found
- ✓ No vulnerable return values found
- ✓ No uninitialized local variables found
- ✓ No default function responses found
- ✓ No missing arithmetic events found
- ✓ No missing access control events found
- ✓ No redundant true/false comparisons found
- ✓ No state variables vulnerable through function calls found
- ✓ No buggy low-level calls found
- ✓ No expensive loops found
- ✓ No bad numeric notation practices found
- ✓ No missing constant declarations found
- ✓ No missing external function declarations found
- ✓ No vulnerable payable functions found
- ✓ No vulnerable message values found



Vulnerability Scan

REENTRANCY

✓ No reentrancy risk found

Severity

Minor

Confidence Parameter

Certain

✓ **External Dependency Risk:** The Givest ecosystem interacts with external infrastructure including Uniswap and Robinhood Chain components.

```
interface IERC20 {
    function transfer(address to, uint256 amount) external returns (bool);
    function transferFrom(address from, address to, uint256 amount) external
returns (bool);
    function balanceOf(address account) external view returns (uint256);
    function approve(address spender, uint256 amount) external returns
(bool);
}

interface IWETH is IERC20 {
    function deposit() external payable;
}

interface ISwapRouter {
    struct ExactInputParams {
        bytes path;
        address recipient;
        uint256 amountIn;
        uint256 amountOutMinimum;
    }

    function exactInput(ExactInputParams calldata params) external payable
returns (uint256 amountOut);
}
```

Vulnerability Description

Scanning Line:

Auto Contract Scan

Basic Info

Token Contract Address	0x0188...7777
Owner (Renounced)	0x0000...0000
Total Supply	1B
Issue Platform	flap.sh

Risk Anti whale is No More modifiable

- ✓ Contract Ownership Renounced
- ✓ Contract is open source
- ✓ Owner can not tamper with balance
- ✓ Doesn't look like a proxy contract
- ✓ No whitelist
- ✓ No blacklist
- ✓ Admin privileges abandoned
- ✓ Can not Mint
- ✓ Can not take back ownership
- ✓ No trading-cool-down mechanism

Mechanism Introduction

Buy Tax 1.5%

Marketing Fee 1.5%

Sell detection 1%

Count 652 1%

0xb6b5...b523

Wallets	Success	Failed	Siphoned
652	652	0	0
Tax	Ave Tax 0% Tax 0%	Count 652	

Contract Creator Address:

0x0188DA44dCc9b6C9D0d80DE904c633c5ff227777

Audited Files

\$GIVEST.Sol

Contracts Creator Hash:

TXN HASH

0x9dbd601e7d45fd19770840ec960fe38cf4977fa471ca462221420d81b
0028e070

Contracts:

Contract Address:

BGOAT: 0x0188DA44dCc9b6C9D0d80DE904c633c5ff227777

MANUAL REVIEW

GIVEST

Hold \$GIVEST. Send for less.

\$GIVEST is not a points system. It is the fee switch for the whole protocol, read onchain on every send.

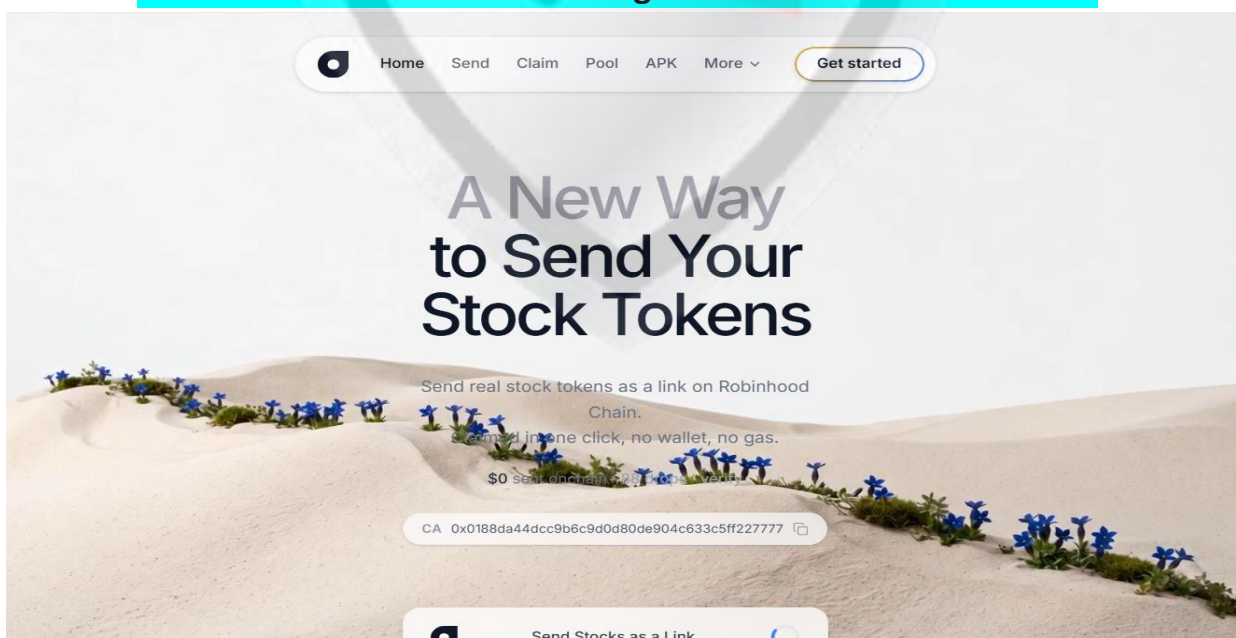
TOKEN NAME: GIVEST

Ticker: \$GIVEST

Chain/Standard: ROBINHOOD NETWORK

LAUNGUGE: SOLIDITY

The GIVEST Platform Is Launching On the Robinhood Network





ISSUES CHECKING STATUS

Issue Description

Checking Status

1.	Compiler errors.	PASSED
2.	RaceConditionsand reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay InData Delivery.	PASSED
4.	Oraclecalls.	PASSED
5.	Front Running.	PASSED
6.	SOLDependency.	PASSED
7.	Integer OverflowAnd Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	DosWith Block GasLimit.	PASSED
10.	Methodsexecutionpermissions.	PASSED
11.	Economy Model of thecontract.	PASSED
12.	TheImpact Of ExchangeRateOnthe MoveLogic.	PASSED
13.	Privateusedata leaks.	PASSED
14.	MaliciousEvent log.	PASSED
15.	Scopingand Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	DesignLogic.	PASSED
19.	Cross-Function raceConditions	PASSED
20.	Save Upon Move contract Implementationand Usage.	PASSED
21.	Fallback FunctionSecurity	PASSED

AUDIT RESULT

PASSED

SMARTCONTRACT AUDIT OF GIVEST



Identifier	Definition	Severity
CEN-02	Initial asset distribution	Minor \$ 

```

contract StockDrops is IUnlockCallback {
    using BalanceDeltaLibrary for BalanceDelta;

    struct Drop {
        address sender;
        address token;
        /// @dev Remaining tokens still in escrow.
        uint128 amount;
        /// @dev Fixed payout per successful claim.
        uint128 amountPerClaim;
        uint40 expiresAt;
        /// @dev Unix timestamp when claiming opens. Use
        `block.timestamp` (or earlier) for instant drops.
        uint40 claimableAt;
        uint16 maxClaims;
        uint16 claimsMade;
        uint8 status; // 0 = none, 1 = active, 2 = fully claimed,
        3 = refunded;
    }

```

Alleviation:

The Distribution asset was acknowledged and ultimately discarded by the **GIVEST** team due to Earning severity. We consider the exhibit fully attended to as it doesn't impose any meaningful security concerns.

RECOMMENDATION

Clarify intent. If anti-bot, consider using time-based or unique buyer count instead.



RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-09 **CENTRALIZEDPRIVILEGES for a detailed understanding.**

ALLEVIATION

The **GIVEST project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project**





CERTIFICATE BY **VITAL BLOCK SECURITY**

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

```

external {
    uint16 bps = feeBpsFor(msg.sender);
    (uint256 net, uint256 feeAmt) =
    splitGross(amount, bps);
    if (net == 0) revert ZeroAmount();
    _requireBalanceIncrease(token,
    amount);
    if (feeAmt > 0) {
        _safeTransfer(token, feeRecipient,
        feeAmt);
        emit ProtocolFeePaid(msg.sender,
        token, feeRecipient, feeAmt, bps
    
```

GIVEST

A New Way to Send Your
Stock Tokens Send real
stock tokens as a link on
Robinhood Chain.

www.usegivestapp

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

 **VITALBlock**
security.

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

CERTIFICATE of COMPLIANCE

This certificate is presented to

- This Project Smart Contract Code Has Been Certified By Vital Block Security
- This Safety Certificate Is Only Valid For >>>>>>>>



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



GIVEST

SCORE

97

MAXIMUM SCORE ACHIEVED

MAKING LIFE EASIER * EASIER EVERYDAY

 **VITALBlock**
security.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor 

Smart contract is interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.



RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.



DISCLAIMERS

Vital Block provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way



to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT'S OR ANY OTHER INDIVIDUAL'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.



LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.



ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, Ulexperts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 4+ casual contributors.

Website: <https://Vitalblock.org>

Email: info@vitalblock.org

GitHub: <https://github.com/vital-block>

Telegram (Engineering): https://t.me/vital_block

Telegram (Onboarding): https://t.me/vitalblock_cmo





Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU**.

MADE IN CANADA



@VB_Audit



Vitalblock.org



@Vitalblock